

**СИСТЕМА ОПРЕДЕЛЕНИЯ «НА МЕСТЕ / НЕ НА МЕСТЕ» И
МОНИТОРИНГА ФОКУС-СЕССИЙ С ИСПОЛЬЗОВАНИЕМ НЕЙРОСЕТЕВОЙ
ДЕТЕКЦИИ В РЕАЛЬНОМ ВРЕМЕНИ**

Донцов И.А.

студент кафедры ППИ ИИТ РТУ МИРЭА

Закожурников С.С.

к.т.н., доцент кафедры ВМ-3 ИПТИП РТУ МИРЭА

***Аннотация.** Предлагается локальная система мониторинга рабочего положения пользователя по данным веб-камеры, реализующая детекцию человека (класс person) в реальном времени, оценку его «вхождения в рабочую зону» и учёт длительности фокус-сессий. В основе решения — компактная нейросеть (YOLOv8n → ONNX) в связке с OpenCV DNN, многопоточной обвязкой на C++ и логикой конечного автомата для устойчивой работы на CPU с опциональной поддержкой CUDA. Система генерирует визуальные и звуковые оповещения при нарушении позиции и сохраняет данные для последующего статистического анализа.*

***Ключевые слова** компьютерное зрение, детекция человека, нейронные сети, YOLOv8, OpenCV, мониторинг присутствия, обработка видеопотока, on-device обработка, приватность данных, временная фильтрация, информационные технологии.*

**REAL-TIME “IN PLACE / OUT OF PLACE” DETECTION SYSTEM AND FOCUS
SESSION MONITORING USING NEURAL NETWORK–BASED HUMAN DETECTION**

Dontsov I.A.

student, Department of Information and Intelligent Technologies of MIREA – Russian
Technological University

Zakozhurnikov S.S.

Candidate of Technical Sciences, Associate Professor, Higher Mathematics – 3 Department, IPTIP
of MIREA – Russian Technological University

***Abstract.** This paper presents a local system for monitoring a user’s working position based on webcam data. The system performs real-time human detection (class person), evaluates whether the user remains within a predefined working zone, and tracks the duration of focus sessions. The proposed solution is based on a compact neural network model (YOLOv8n exported to ONNX) integrated with OpenCV DNN. The system is implemented in C++ with a multithreaded architecture and a finite-state machine to ensure stable real-time operation on CPU, with optional CUDA acceleration. Visual and audio notifications are generated when the user leaves the designated area, and the collected data can be used for further statistical analysis of focus behavior. The system operates fully on-device, ensuring user privacy without transmitting video data externally.*

Keywords:

computer vision, human detection, neural networks, YOLOv8, OpenCV, presence monitoring, video stream processing, on-device processing, data privacy, temporal filtering, information technologies.

Современная экономическая и научная деятельность [1-8] всё больше опирается на длительную работу за компьютером. Нарушения позы, частые отвлечения и нерегламентированные перерывы снижают продуктивность и ухудшают здоровье сотрудников. Инструменты, автоматически фиксирующие «на месте / не на месте» и измеряющие продолжительность непрерывной фокусной работы, актуальны для исследований эргономики, оценки производительности и разработки рекомендаций по тайм-менеджменту. При этом важно обеспечивать низкую задержку, конфиденциальность и пороговую чувствительность под конкретные условия (камера, освещение, угол обзора) — что требует сочетания компьютерного зрения и математических моделей временного сглаживания и детекторной уверенности.

1) Загрузка модели YOLO8n в переменную net для работы с нейросетями

Передаем модель в формате .onnx в dnn::net из файла `PATH_TO_MODEL`.

Если доступно графическое ускорение CUDA, то включаем его через `net.setPreferableBackend`.

2) Функция для подготовки к работе с моделью `preperingModel`.

Листинг 1.

Выполняем первичную обработку изображения, которое идет на вход модели и загружаем его через `net.setInput()`

```
cv::Mat imgBGR, imgResized;  
cv::cvtColor(img, imgBGR, cv::COLOR_BGR2BGR);  
cv::resize(imgBGR, imgResized, cv::Size(inpW, inpH), 0, 0, cv::INTER_LINEAR);  
cv::Mat blob = cv::dnn::blobFromImage(imgResized, 1 / 255.0, cv::Size(inpW, inpH),  
cv::Scalar(0, 0, 0), true, false);  
net.setInput(blob);
```

Листинг 2.

Получаем вывод из нейросети в вектор `outputs`.

Меняем порядок столбцов в матрице вывода и транспонируем (это нужно для корректного порядка выводов, с которым будем удобно работать)

```
std::vector<cv::Mat> outputs;  
net.forward(outputs, net.getUnconnectedOutLayersNames());  
cv::Mat detMat = outputs[0];  
detMat = detMat.reshape(1, detMat.size[1]).t();
```

Проходим по всем строкам в матрице вывода и каждую отдельную строку рассматриваем как `float* data`.

После этого создаем матрицу со всеми оценками вероятностей, для каждого класса, который есть в нейросети `cv::Mat scores(1, detMat.cols - 4, CV_32FC1, data + 4)`; (нас интересует только класс “человек”, этим мы займемся дальше).

Далее фильтруем все выводы оценки нейросети по классу `personClassId` и по порогу

`const float confThreshold = 0.7f`. Условно это значит, что мы просто убираем все детекции, в которых модель не уверена на 70%+.

Далее получаем `x`, `y` и высоту с шириной той детекции, которая прошла проверку и отправляем ее в вектор `std::vector<Detection> detections`;

Он хранит в себе структуру с `id` класса детекции, с уверенностью в ней, координатами для отрисовки и структуру `Distance` с которой мы будем работать дальше.

3) Функция для отрисовки детекций на изображении `drawBoxes`.

Создаем вектор для хранения боксов, которые мы будем рисовать - `std::vector<cv::Rect> boxes`;

вектор для хранения уверенностей в этих боксах - `std::vector<float> confidences`.

Далее проводим финальные проверки и фильтрацию через `NMSBoxes` по порогу `nmsThreshold = 0.5f` и наконец получаем индексы всех детекций, которые прошли проверки в вектор `std::vector<int> indices`.

Далее рисуем все боксы на изображении, которое подавалось в нейросеть через `cv::rectangle(img, d.box, cv::Scalar(0, 0, 255), 2)`;

И добавляем текст с итоговой уверенностью и элементами для определения дистанции через `cv::putText(img, cv::format("Height: %d", d.distance.height), cv::Point(d.box.x + 270, d.box.y - 10), cv::FONT_HERSHEY_SIMPLEX, 0.5, cv::Scalar(255, 0, 0), 1);`.

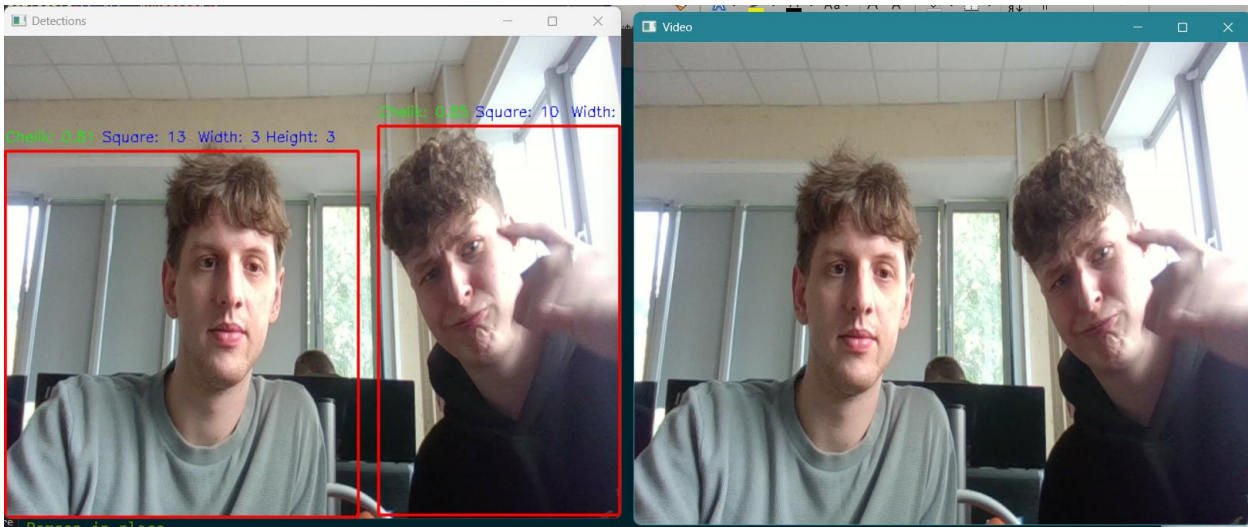


Рисунок 1. Результат работы нейросети

3) Создание таймера для фиксации времени фокуса

Сделана простая функция для фиксации человека на рабочем/учебном месте, теперь нужен таймер для подсчета времени фокусировки.

Создадим `class Timer`, он и будет выполнять функцию подсчета времени.

Таймер будет запускаться как отдельный поток с атомарными переменными `std::atomic<int> seconds{ 0 }` для подсчета времени и флагом `std::atomic<bool> running{ false }`, который будет определять работает ли сейчас таймер.

3.1) Функция start для запуска таймера

Листинг 3.

```
void Timer::start()
{
    bool expect = false;
    if (!running.compare_exchange_strong(expect, true)) return;

    worker = std::thread([this]()
    {
        while (running.load())
        {
            std::this_thread::sleep_for(std::chrono::seconds(1));
            if (!running.load()) break;
            seconds.fetch_add(1);
        }
    });
}
```

Проверим работает ли таймер через `compare_exchange_strong(expect, true)`.

В случае, если таймер уже запущен, для предотвращения дублирования потока просто выходим из функции.

Если поток еще не работает – запускаем его и поддерживаем пока `running == true`.

3.2) Функция stop для остановки таймера

Листинг 4.

```
void Timer::stop()
{
    bool expect = true;
    if (!running.compare_exchange_strong(expect, false)) return;
    if (worker.joinable())
        worker.join();
}
```

Через тот же `compare_exchange_strong(expect, false)` проверяем был ли уже остановлен таймер. Если да – просто выходим, если нет – останавливаем через `worker.join()` и выходим.

4) Основная логика и измененная функция `inPlaceOrNot`.

Усилим логику основной функции обнаружения человека, т.к. простая реализация по типу:

```
if (d.distance.square < DISTANCE_THRESHOLD)
{
    std::cout << "Person not in place!" << std::endl;
    return false;
}
else
{
    std::cout << GREEN << "Person in place." << STANDART << std::endl;
    return true;
}
```

не подходит.

Во-первых, условие “площадь обнаружения меньше допустимой - человек ушел” может работать некорректно. Например, человек повернулся боком на несколько секунд или наклонился, тогда условие `d.distance.square < DISTANCE_THRESHOLD` сработает.

Во-вторых, возможны шумы самой модели YOLO. Она может пропускать какие-то кадры или просто не находить там человека (такое иногда случается, например, из-за света).

И наконец человек может просто отвлечься на пару секунд, а система сразу определит, что он ушел и завершит сессию.

Сделаем проверку сразу по трем условиям: площадь, высота и ширина бокса.

Листинг 5.

```
bool isInPlace() const
{
    return (square >= SQUARE_DISTANCE_THRESHOLD &&
            width >= WIDTH_DISTANCE_THRESHOLD &&
            height >= HEIGHT_DISTANCE_THRESHOLD);
}
```

Функция будет возвращать `true` только если все 3 условия соблюдены.

Далее сделаем фильтр шумов.

Листинг 6.

```
if (inPlace)
{
    consecutiveInPlace++;
    consecutiveNotInPlace = 0;
}
else
{
    consecutiveNotInPlace++;
    consecutiveInPlace = 0;
}
bool stableInPlace = (consecutiveInPlace >= framesForTrigger);
bool stableNotInPlace = (consecutiveNotInPlace >= framesForTrigger);
```

Логика проста: если человек на месте, накапливаем кадры его детекции, если человек не на месте, тогда накапливаем кадры его отсутствия.

Если кадров больше чем `framesForTrigger` – тогда меняем его состояние на `stableInPlace` или на `stableNotInPlace`.

Осталась последняя проблема, требующая самого подробного решения.

Сделаем 3 разных состояния: человек на месте, отошел (но может вернуться), ушел (конец сессии).

Листинг 7.

```
enum class PlaceStatus
{
    IN_PLACE, // НА МЕСТЕ
    LEFT_THE_PLACE, // ПОКИНУЛ МЕСТО ПОЛУЧАЕТ ПРЕДУПРЕЖДЕНИЕ
    OUT_OF_PLACE // ВНЕ МЕСТА
};
```

По умолчанию считаем, что человек сразу на месте и устанавливаем

```
PlaceStatus placeStatus = PlaceStatus::IN_PLACE;
```

Запускаем таймер начала сессии и ждем изменения состояния на LEFT_THE_PLACE

Листинг 8.

```
switch (placeStatus)
{
    case PlaceStatus::IN_PLACE:
    {
        if (!timer.isRunning()) timer.start();

        if (stableNotInPlace)
        {
            placeStatus = PlaceStatus::LEFT_THE_PLACE;
            warningStartTime = now;
            warningShown = false;
            std::cout << RED << "LEFT THE PLACE: WARNING STARTED" << STANDART
<< std::endl;
        }
        break;
    }
}
```

В этом кейсе мы обрабатываем состояние IN_PLACE.

Если срабатывает флаг stableNotInPlace сразу переводим состояние в LEFT_THE_PLACE, отмечаем время когда человек покинул рабочее место warningStartTime = now, начинаем обработку следующего состояния.

Листинг 9.

```
case PlaceStatus::LEFT_THE_PLACE:
{
    if (stableInPlace)
    {
        placeStatus = PlaceStatus::IN_PLACE;
        player.stop();
        alarmPlaying = false;
        break; }

    if (!alarmPlaying)
    {
        player.play();
        alarmPlaying = true; }

    auto elepsed = std::chrono::duration_cast<std::chrono::seconds>(now -
warningStartTime);
    if (elepsed > warningDurationFirst && elepsed <= warningDurationSecond &&
!warningShown)
    {
        ShowBalloon(L"Warning", L"You have left the designated area. Please
return.");
        warningShown = true; }

    if (elepsed > warningDurationSecond)
    {
        placeStatus = PlaceStatus::OUT_OF_PLACE;
        int focusSeconds = timer.elepsed();
        {
            std::wstringstream ss;
            ss << L"Focus session length: " << focusSeconds << L" s";
            ShowBalloon(L"Focus ended", ss.str().c_str());
        }
        timer.reset(); }

    break;}
}
```

В этом кейсе обрабатываем состояние LEFT_THE_PLACE.

Сразу же проверяем вернулся ли человек. Если вернулся – переводим статус назад в IN_PLACE, останавливаем все предупреждения и выходим.

Если человек все же не вернулся включаем звуковую сирену:

```
if (!alarmPlaying)
{
    player.play();
    alarmPlaying = true; }
}
```

Листинг 9.1

```
void Player::play()
{
    std::wstring cmd = L"open \"" + soundToPlay + L"." type mpegvideo alias mp3";
    mciSendStringW(cmd.c_str(), NULL, 0, NULL);
    mciSendStringW(L"play mp3 repeat", NULL, 0, NULL);}
```

Функция откроет mp3 файл и через Media Control Interface запустит его и зациклит до момента пока мы его принудительно не выключим.

Листинг 9.2

```
void Player::stop()
{
    mciSendStringW(L"stop mp3", NULL, 0, NULL);
    mciSendStringW(L"close mp3", NULL, 0, NULL);}

```

Выключаем плеер.

Далее проверяем сколько времени прошло с момента нашей отметки о том, что человек ушел:

```
auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(now -
warningStartTime);

```

Выводим предупреждение прямо в систему:

```
if (elapsed > warningDurationFirst && elapsed <= warningDurationSecond &&
!warningShown)
{
    ShowBalloon(L"Warning", L"You have left the designated area. Please
return.");
    warningShown = true;
}

```

Предупреждение – это системное уведомление, которое выглядит вот так:

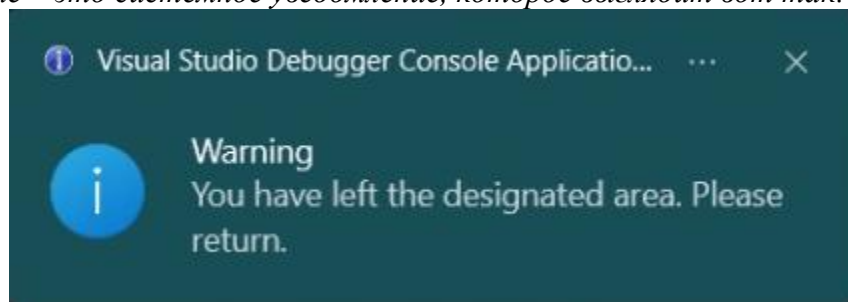


Рисунок 2. Системное уведомление

Типичное системное уведомление winAPI. Из интересного - мы удаляем уведомление в отдельном потоке, чтобы не блокировать основной.

После предупреждения, если человек не вернулся, сработает условие `if (elapsed > warningDurationSecond)`, что время вышло.

Переводим состояние в OUT_OF_PLACE, получаем время фокусировки

`int focusSeconds = timer.elapsed()`, выводим уведомление о том, что сессия кончилась и о ее длительности, обнуляем таймер.

Листинг 10.

```
case PlaceStatus::OUT_OF_PLACE:
{
    if (alarmPlaying)
    {
        player.stop();
        alarmPlaying = false;
        if (stableInPlace)
        {
            placeStatus = PlaceStatus::IN_PLACE;
            timer.stop();
            timer.reset();
            ShowBalloon(L"Welcome back", L"You are back in the designated area.");
        }
        break;
}

```

Выключаем сирену и ждем флага `stableInPlace`, чтобы снова запустить сессию.

При работе над проектом выполнено:

- подготовка и экспорт модели YOLOv8n в формат ONNX для использования с OpenCV DNN;
- реализация предобработки и корректного преобразования выхода модели (reshape / transpose, parsing детекций, NMS);
- разработка и реализация state-machine (PlaceStatus) с пороговой логикой для сглаживания;
- реализация таймера фокус-сессий (Timer) с безопасной многопоточной работой (atomic/condition_variable) и API start/stop/reset/elapsed;

- создание Player и TrayNotifier с thread-safe очередью задач, обеспечивающих ненадёжное и неблокирующее уведомление пользователя (балун + звуковой сигнал);
- тестирование системы на наборе изображений, отладку масштабирования bbox → физические размеры (приблизённое оценивание расстояния по площади бокса) и проверку поведения при различных сценариях (короткие уходы, длительные уходы, возвращения).

Перспективы использования полученных результатов:

- Рабочие станции и офисы: интеграция как помощник по тайм-менеджменту (фокус-сессии, напоминания о перерывах), улучшение эргономики рабочих мест.
- Исследования эргономики и продуктивности: сбор анонимной статистики о длительности фокус-сессий, частоте прерываний и корреляции с результативностью, что полезно для исследований человеческого фактора.
- Обучающие системы и self-tracking: персональные инструменты для повышения концентрации (pomodoro-style), адаптивные уведомления в зависимости от профиля пользователя.
- Интеграция с корпоративными системами: экспорт логов и аналитики в BI-инструменты для агрегированного анализа.

Список использованной литературы:

1. Yiyang Liu, Jiaxing Li, Cuiping Zhang, Hongqiang Rua, Wei Wang, Xinyan Yue, SiC whisker enhanced reaction bonded boron carbide composites by silicon infiltration: The effect of SiC whisker contents, *Ceramics International*, 2025, <https://doi.org/10.1016/j.ceramint.2025.10.435>.
2. Guogang Wang, Erlei Bai, Tao Chen, Zhigang Gao, He Huang, Wei Xia, Experimental study on dynamic compressive properties of heated silicon carbide modified concrete, *Construction and Building Materials*, Volume 489, 2025, 142270, <https://doi.org/10.1016/j.conbuildmat.2025.142270>.
3. Кузеванов, В. С. Модель тепломассопереноса в печах при производстве карбида кремния / В. С. Кузеванов, Г. С. Закожурников, С. С. Закожурников // *Международный научный журнал Альтернативная энергетика и экология*. – 2015. – № 7(171). – С. 75-81. – DOI 10.15518/isjaee.2015.07.006. – EDN TUBQUX.
4. Кузеванов, В. С. Оптимизация процесса плавки карбида кремния с целью повышения ее производительности и снижения расхода электроэнергии / В. С. Кузеванов, С. С. Закожурников, А. Б. Гаряев // *Промышленная энергетика*. – 2015. – № 6. – С. 29-33. – EDN UEAGQD.
5. Гаряев, А. Б. Модель осадки шихты при производстве карбида кремния / А. Б. Гаряев, С. С. Закожурников, В. С. Кузеванов // *Промышленная энергетика*. – 2016. – № 9. – С. 27-31. – EDN WWCMYR.
6. Model of continuous production of fine silicon carbide / V. S. Kuzevanov, A. B. Garyaev, S. S. Zakozhurnikov, G. S. Zakozhurnikova // *IOP Conference Series: Materials Science and Engineering : International Workshop "Advanced Technologies in Material Science, Mechanical and Automation Engineering – MIP: Engineering – 2019"*, Krasnoyarsk, 04–06 апреля 2019 года. Vol. 537. – London: Institute of Physics and IOP Publishing Limited, 2019. – P. 32106. – DOI 10.1088/1757-899X/537/3/032106. – EDN LMVGJD.
7. Finely dispersed silicon carbide synthesis model in the electrothermal reactor with periodic batch loading / V. S. Kuzevanov, S. S. Zakozhurnikov, G. S. Zakozhurnikova, A. B. Garyaev // *Journal of Physics: Conference Series* : 3, Moscow, 19–23 октября 2020 года. – Moscow, 2020. – P. 022054. – DOI 10.1088/1742-6596/1683/2/022054. – EDN RVGPGV.
8. Kuzevanov, V. S. Model and results of a study of the synthesis of finely dispersed silicon carbide in an electro-thermal reactor / V. S. Kuzevanov, S. S. Zakozhurnikov, G. S. Zakozhurnikova // *Solid State Phenomena*. – 2021. – Vol. 316. – P. 147-152. – EDN

XNOUYB.